

# 织物缺陷检测：从数据难点到方法选择

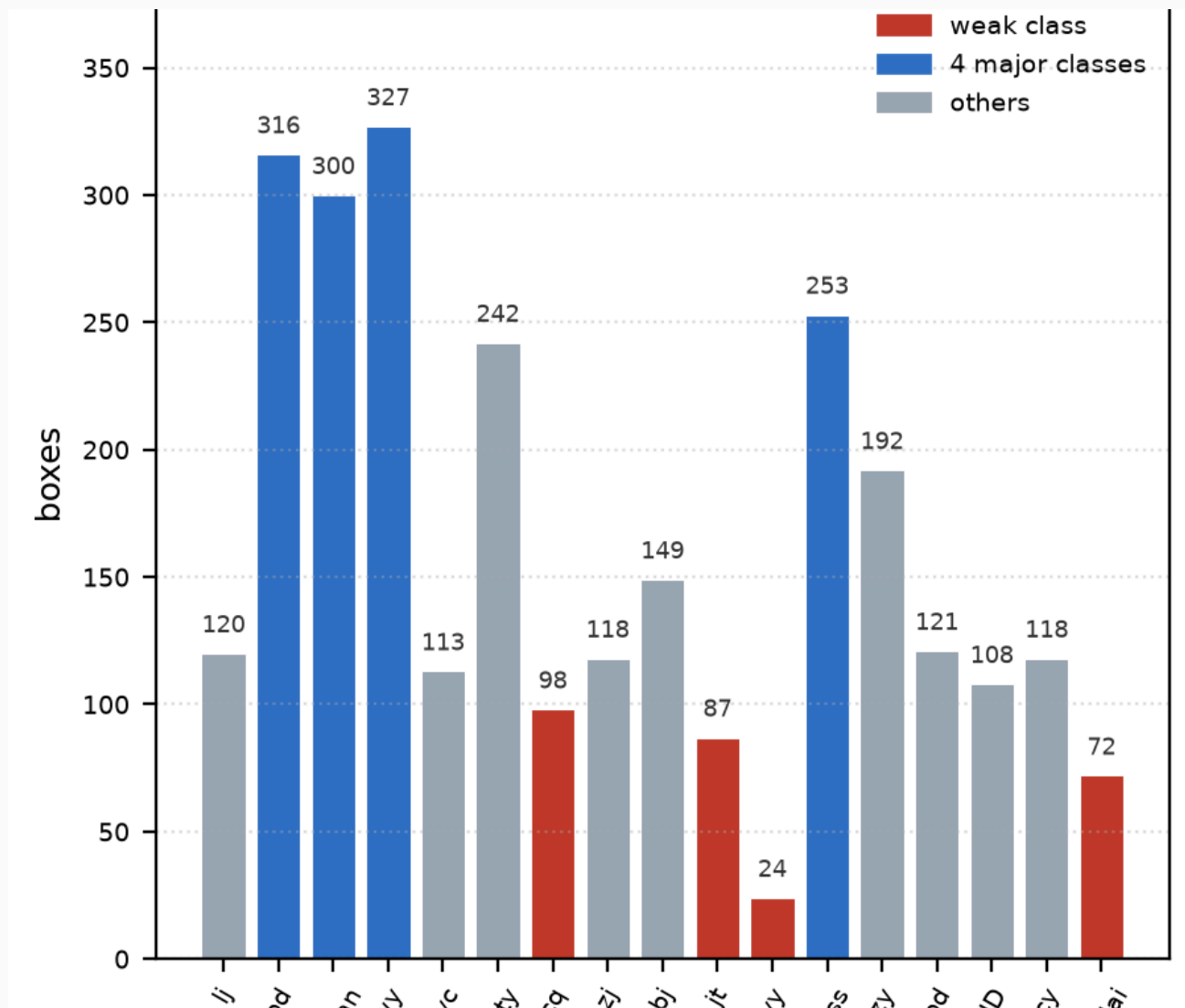
17 类缺陷 · 2048×2048 原图 · RT-DETR 基线对比

---

大学生竞赛素材 · 2026-09

## 任务定义

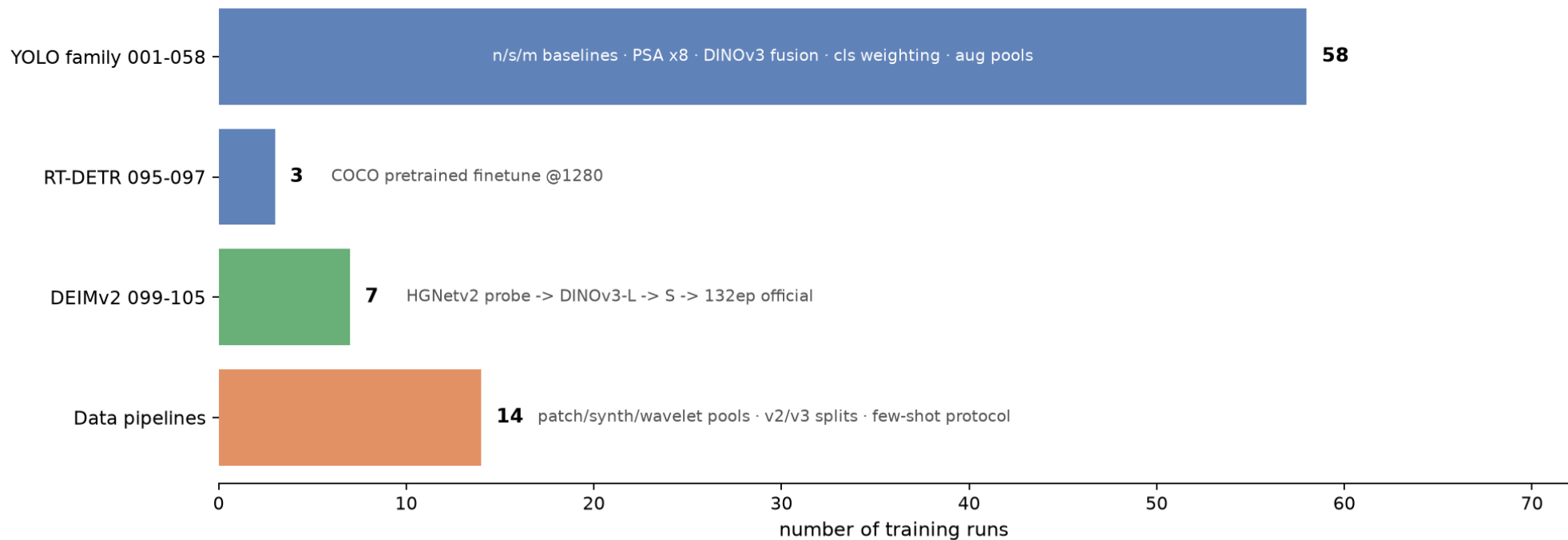
- 织物表面缺陷检测：17 类，框级定位 (bbox)
- 原图  $2048 \times 2048$ ，训练 1445 / 验证 181 / 测试 181
- 推理分辨率 1280 (640/1280 训练模型不能混用)
- 只提交训练权重，禁用 TTA / 集成等推理端手段



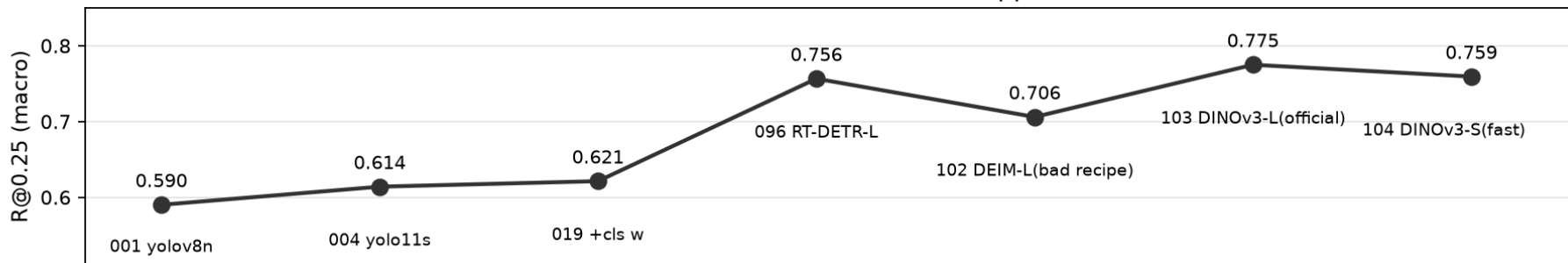
## 实验全景：工作量与方法演进



Workload: 81 training runs + 6 data pipelines + 3 eval protocols



Metric evolution: 3 architecture rounds, +11.6pp (0.590 -> 0.775)



81 个训练 run + 6 条数据管线（patch/合成/小波池、v2/v3 无泄漏划分）+ 3 套评测协议；三轮架构迭代将  $R@0.25$  从 0.590 推进至 0.775

# 负结果：我们排除了什么（以及为什么）

## 负结果：我们排除了什么（以及为什么）

|                                                                                                 |                           |
|-------------------------------------------------------------------------------------------------|---------------------------|
| <b>Attention structures (PSA/C2PSA/SE)</b>                                                      | 6 runs (009-017)          |
| all BELOW yolo11s baseline (-0.03 ~ -0.18): attention adds no signal on this data               |                           |
| <b>Background suppression</b>                                                                   | 4 forms (065/066/067/070) |
| all negative: hard-neg backgrounds hurt bd/zmty; high FP != pure background (sparse val labels) |                           |
| <b>Oversampled / upscaled pools</b>                                                             | run 078                   |
| 11x train-inference scale mismatch -> bd recall drops; scale alignment beats data volume        |                           |
| <b>Copy-Paste</b>                                                                               | run 031                   |
| 0.595 vs 0.614 baseline - no gain on dense small defects                                        |                           |
| <b>YOLO-Master / LoRA 069</b>                                                                   | parked                    |
| superseded by data-driven route + cloud training                                                |                           |

11 组对照实验证伪 4 条直觉路线——负面结论同样来自实验数据，构成本次工作方法论的验证部分

## 数据难点（三条）

### 难点 1：类别极不均衡 + 部分类样本极少

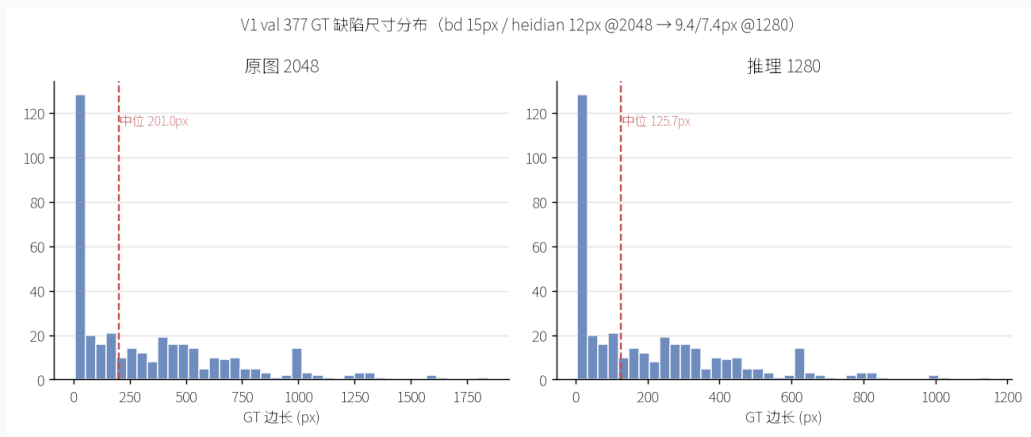
- bd/heidian 是“白点/黑点”，GT 仅 44/39 个实例
- wy 污印 39 / zmt y 37 / lj 12 / yy 压印仅 2 个

### 难点 2：小目标尺度失配

- 中位 bd 15px、heidian 12px (2048)
- 1280 推理下仅 9.4 / 7.4px，小于 40px 特征网格
- mAP50 仅 0.15-0.29（小目标定位差）

### 难点 3：跨类视觉相似 / 漏标注

- heidian→wy、zmt y→bmss 等易混淆
- val 标注稀疏：FP 高  $\neq$  纯背景，压背景会压真阳性



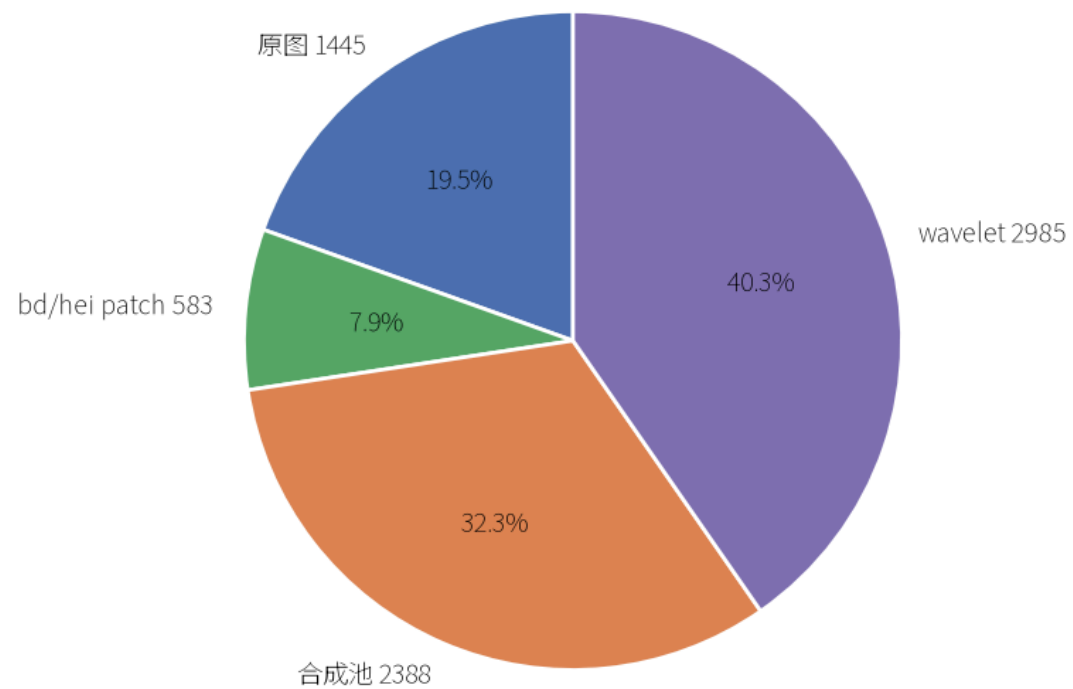
### 数据处理（三步）

[

1. **尺寸对齐**：训练统一 1280，消除 640/1280 尺度失配崩溃
  2. **增强池扩充**：实例中心 patch (bd/hei 583) + 合成池 + wavelet
  3. **无泄漏划分**：8:1:1 图级重划，类分层迭代 (val 每类  $\geq 8$ )
- 教训：跨集验证有 80.9% 泄漏，任何跨划分数值作废

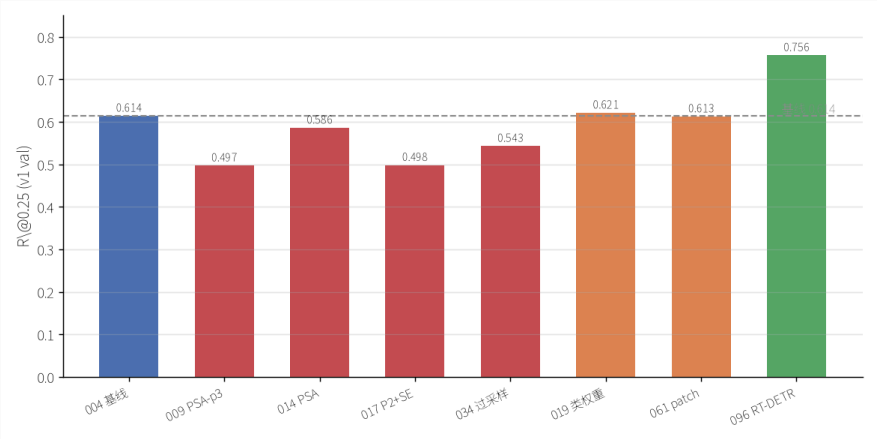
**注**：增强池提升小目标召回，但整体 mAP 收益有限——增强不解决尺度表征 ]

训练数据池构成（v1 系，总 7401 张）



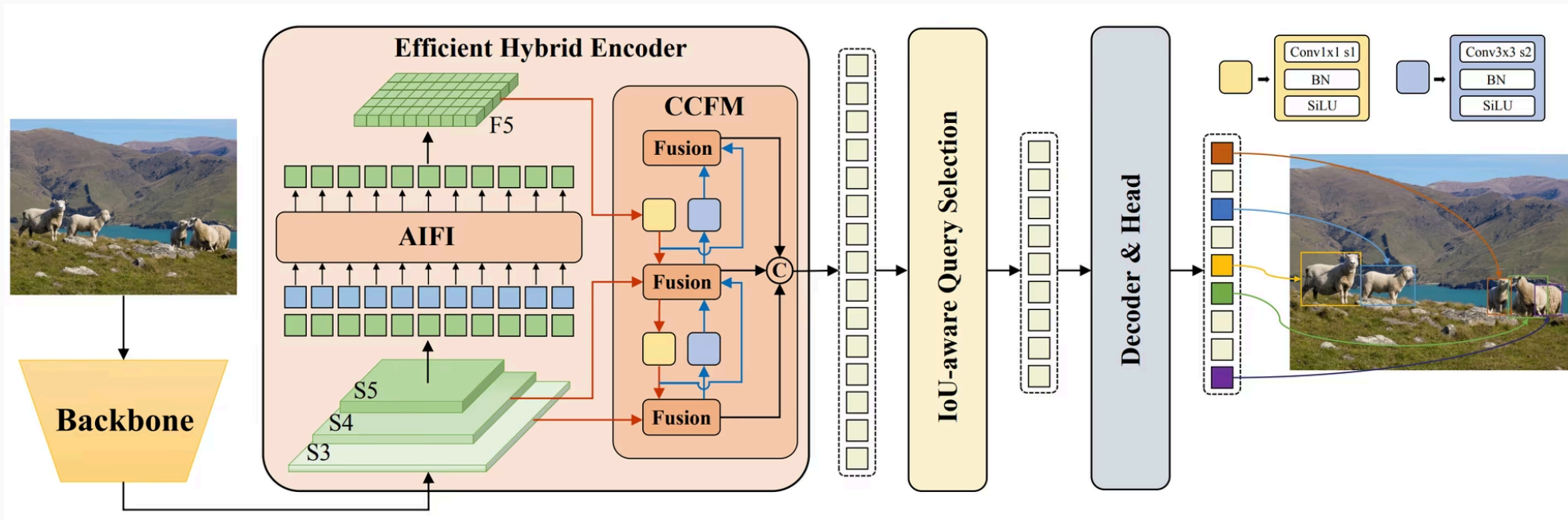
方法对比：YOLO 基线 vs RT-DETR 最优

| 方法              | R@0.25 | mAP50 |
|-----------------|--------|-------|
| yolo11s 基线 004  | 0.614  | 0.605 |
| yolo11s+cls 019 | 0.621  | 0.577 |
| RT-DETR-L 096   | 0.757  | 0.677 |
| RT-DETR-L 097   | 0.716  | 0.636 |



R@0.25 对比 (v1 val)

## RT-DETR 架构与创新点



图源：RT-DETR 官方架构图 (arXiv:2304.08069, 经 Ultralytics 文档引用)；P2 辅助头为本工作新增

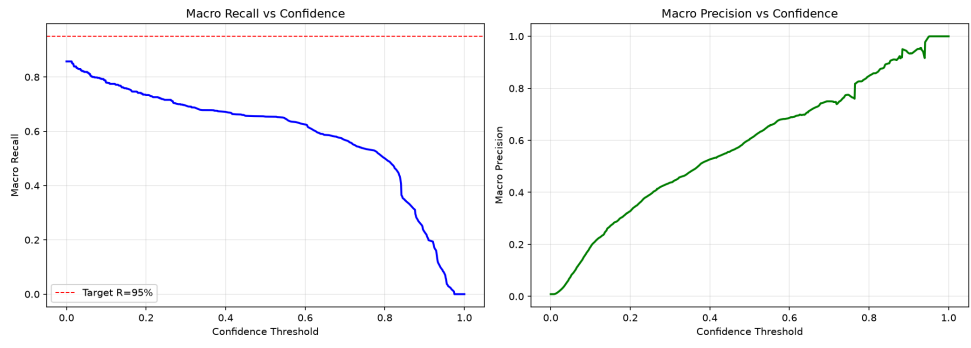
**基线：** HGNetv2 P2-P5 → 混合编码器 (AIFI + 跨尺度融合) → IoU-aware query 选择 → 解码器 (6 层×300 query, P3/P4/P5) → 框 + 类别

**创新：** **P2AuxHead** (P2/4 轻量头, 训练侧, 推理零开销)



RT-DETR 训练配置与效果

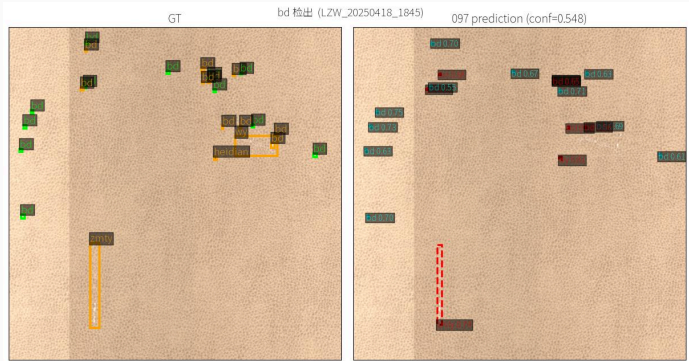
| 项             | 值                              |
|---------------|--------------------------------|
| 模型            | RT-DETR-L 32M, COCO 预训练        |
| 数据            | b-final-v3 (2838 池)            |
| 训练            | 1280, batch8, 72ep, AdamW 1e-4 |
| 口径            | self-val + conf 0.25 + IoU 0.5 |
| R@0.25        | 0.716 (vs YOLO 0.62)           |
| mAP50         | 0.636                          |
| FPS (3060 实测) | 27.4 @1024 / 19.8 @1280 (bs4)  |



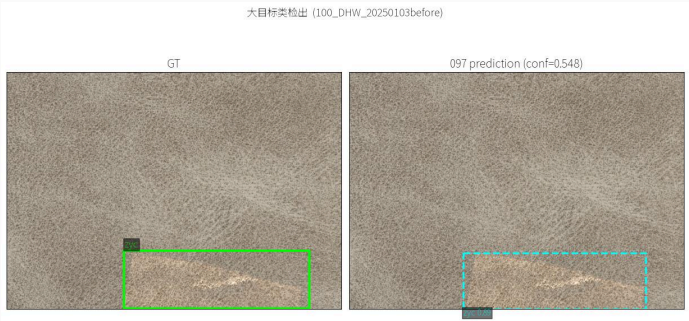
conf-R/P 曲线

# 检出与漏检案例（左 GT / 右 预测）

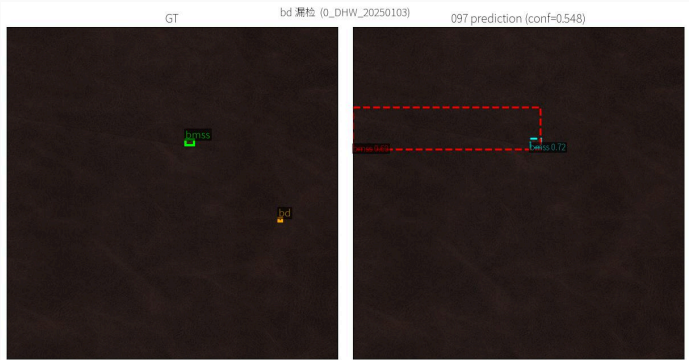
## 检出与漏检案例（左 GT / 右 预测）



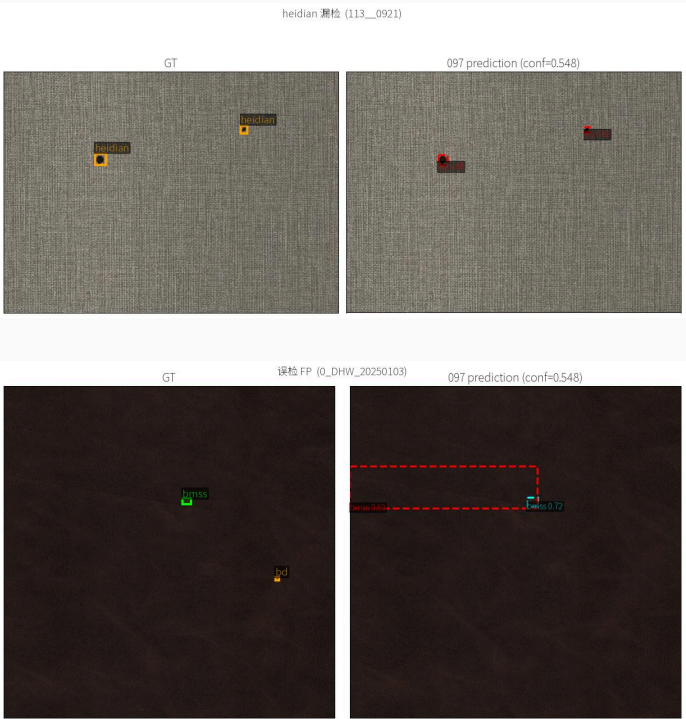
# 检出与漏检案例（左 GT / 右 预测）



bd / heidian 小目标检出 + 大目标检出 (conf 0.55)

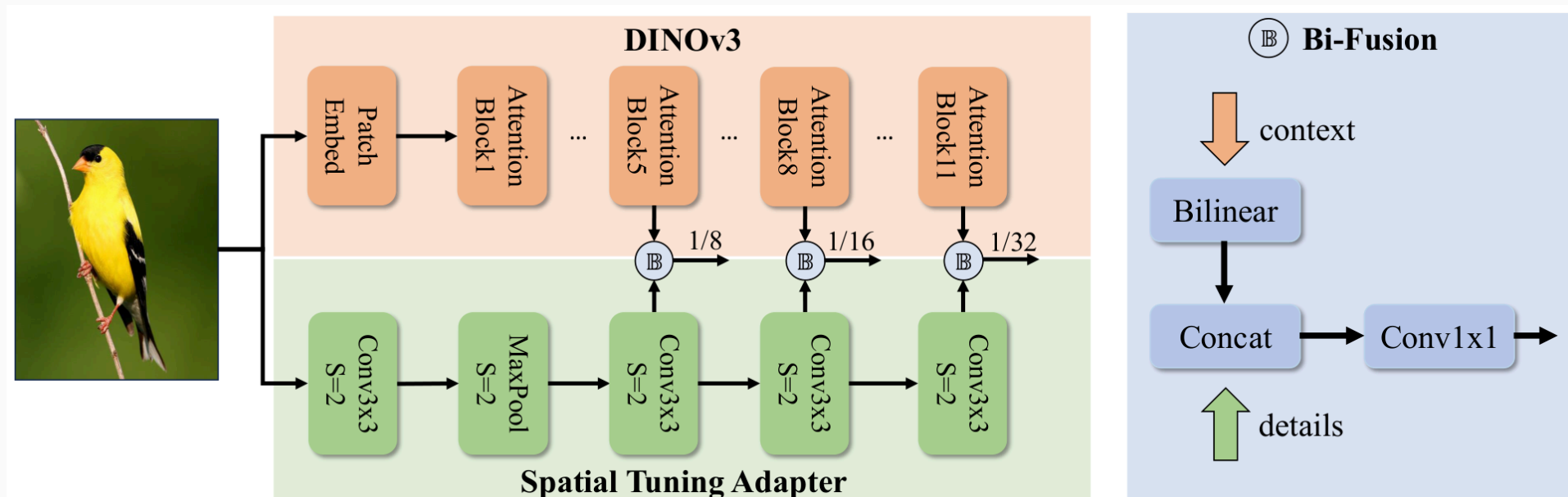


# 检出与漏检案例（左 GT / 右 预测）



漏检（bd/hei 7-20px）与误检——小目标仍是主短板

## DEIMv2: Real-Time Detection Meets DINOv3



图源：DEIMv2 官方论文 arXiv:2509.20787 (Spatial Tuning Adapter 架构图)

**架构路线** (S/M/L/X 档): DINOv3 自监督预训练 ViT → **STA (Spatial Tuning Adapter)** → 混合编码器  
→ 简化解码器 → Dense O2O 训练

**STA 解决什么:** DINOv3 只有**单尺度**输出 (ViT 16×16 patch) → STA 用轻量卷积把 1/8、1/16、1/32 三个尺度的细粒度细节“注入”各层，无需改动预训练 backbone

## 论文三个创新:

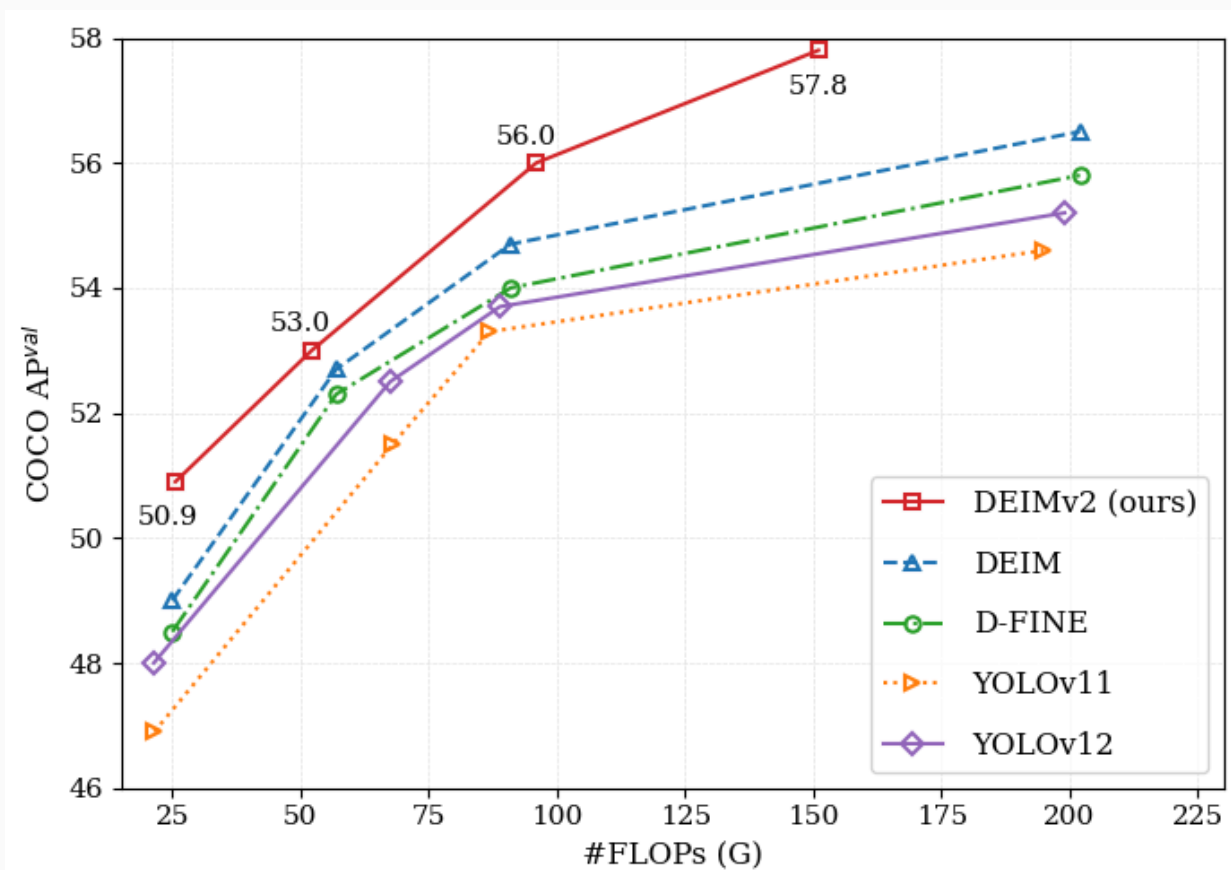
1. **STA 空间调谐适配器**: 单尺度→多尺度, 语义+细节互补
2. **高效解码器**: 4 层×300 query, cross-attn 简化
3. **升级版 Dense O2O**: Mosaic+MixUp+CopyBlend 密集监督

DEIMv2 Model Zoo 与本职工作选型

| 档位 | COCO AP | 参数    | backbone | 适合   |
|----|---------|-------|----------|------|
| N  | 43.0    | 3.6M  | HGNetv2  | 轻量   |
| S  | 50.9    | 9.7M  | DINOv3   | 速度友好 |
| M  | 53.0    | 18.1M | DINOv3   | 均衡   |
| L  | 56.0    | 32.2M | DINOv3   | 本文选择 |
| X  | 57.8    | 50.3M | DINOv3   | 精度极限 |

为什么选 DINOv3-L:

- 织物缺陷是弱纹理细粒度判别 (wy 污印 ↔ zmtly 脏印) ——自监督 DINOv3 表征的强项
- 与 RT-DETR-L 同为 32M 参数档，公平对比
- 从 COCO 预训练权重 -t 微调 (分类头重映射 80→17 类)





训练配方：从踩坑到官方配方

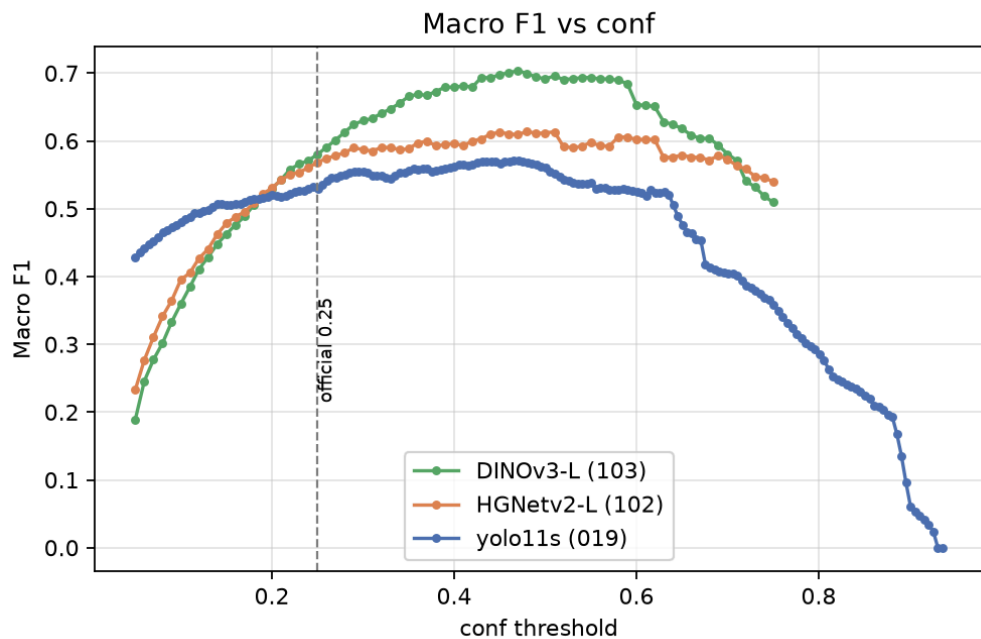
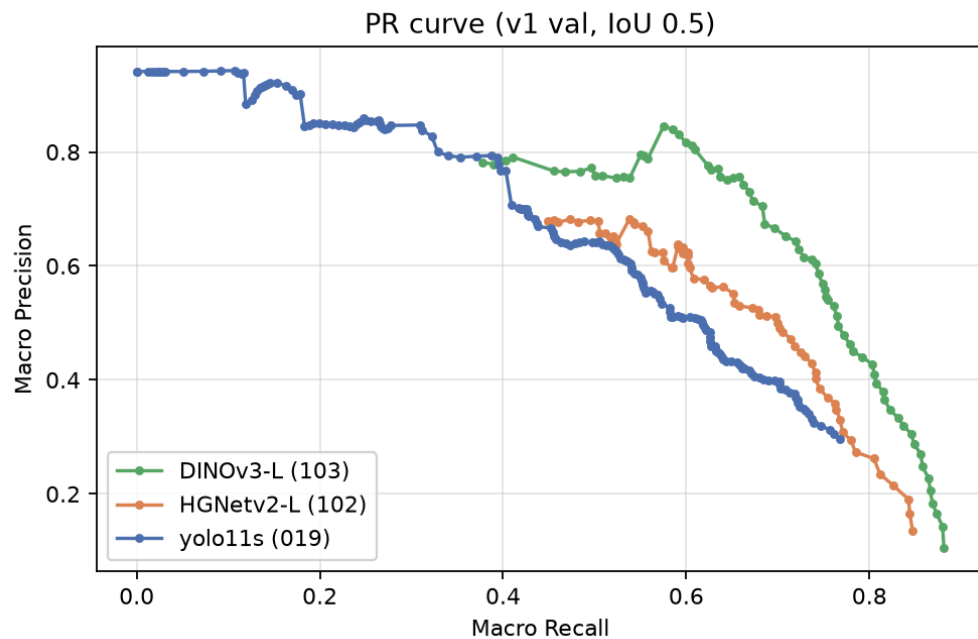
| 项     | 首次尝试 102       | 最终 103            |
|-------|----------------|-------------------|
| lr    | 1e-4（保守）       | 官方 5e-4 分层        |
| 分类头   | 80 类（漏配）       | 17 类              |
| batch | 4              | 4×累积 8=32         |
| 增强    | 无 Mosaic/MixUp | 官方 Dense O2O      |
| 输入归一化 | 缺 Normalize    | ImageNet mean/std |

对照实验证明配方价值：

- 102（同数据，错误配方）：AP50 0.594，R@0.25 0.706
- 103（官方配方）：AP50 **0.708**，R@0.25 **0.775**

结论：SOTA 架构只有配上官方训练配方才能兑现——直接照抄保守参数会浪费 7pp

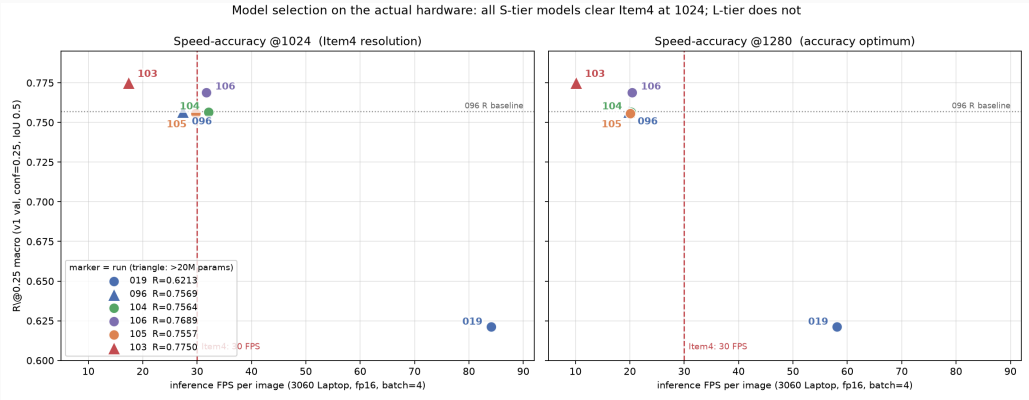
## P-R 曲线：全阈值扫描



同协议扫描 (v1 val, IoU $\geq$ 0.5, conf 0.05-0.94): 103 全曲线压制 102; best-F1 0.614 $\rightarrow$ 0.704, 峰宽 conf 0.41-0.58 (交卷阈值鲁棒)

# 结果：DEIMv2 DINOv3 系列全面领先

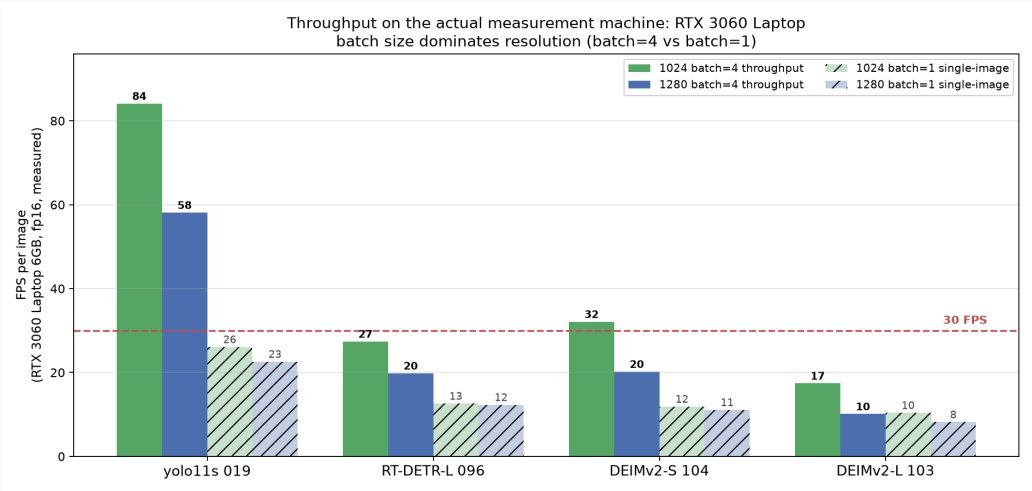
## 结果：DEIMv2 DINOv3 系列全面领先



| 指标       | 103 L  | 104 S  | 096    |
|----------|--------|--------|--------|
| R@0.25   | 0.775  | 0.756  | 0.757  |
| P@0.25   | 0.460  | 0.473  | 0.403  |
| AP50     | 0.708  | 0.696  | 0.676  |
| AP50-95  | 0.500  | 0.498  | 0.462  |
| FPS@1024 | 17.4 ✗ | 32.1 ✓ | 27.4 ✗ |
| FPS@1280 | 10.1   | 20.2   | 19.8   |

FPS = 本机 RTX 3060 Laptop 原生实测 (fp16, batch=4, v1 val 181 图, 纯前向); **104 (S 档 9.7M)** 实现精度与速度双达标: R 超 RT-DETR 基线且 1024 下 32.1 FPS > 30 (Item4); 103 (L 档) 为精度上限, 需工程加速才能过速度线

推理速度 · 口径差点让结论写反



同一模型，两种测法差 1.2-3.2×。

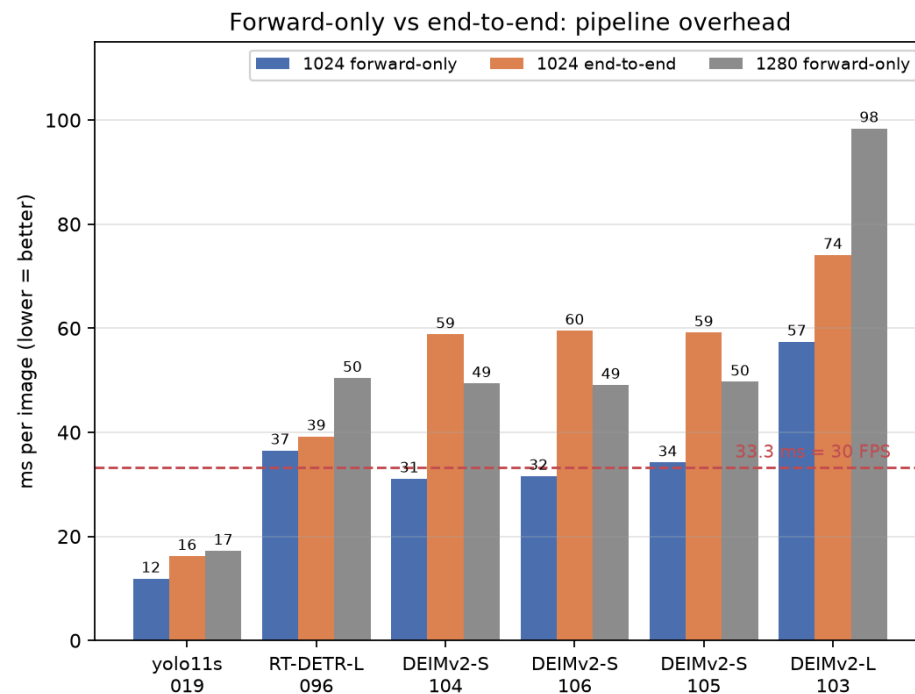
初测按 **batch=1 单图** 计时（模拟“一张图进来”的延迟），得到 104 = 11.1 FPS，据此判断“Item4 擦线不过、需上 TensorRT”。

复核发现：历史速度表用的是 **batch=4**，且部署端必然是持续流而非单图。改为 batch=4 重测，同一权重、同一 val 集、同一机器：

- 104 @1280: 11.1 → **20.2** FPS (2.7×)
- 019 @1280: 22.6 → **58.1** FPS (3.2×)

**单图延迟不是部署指标**：batch=1 时每张图都要付一次 Python/内核启动开销，GPU 空转。

## 推理速度 · 分辨率 vs 批量：两个杠杆

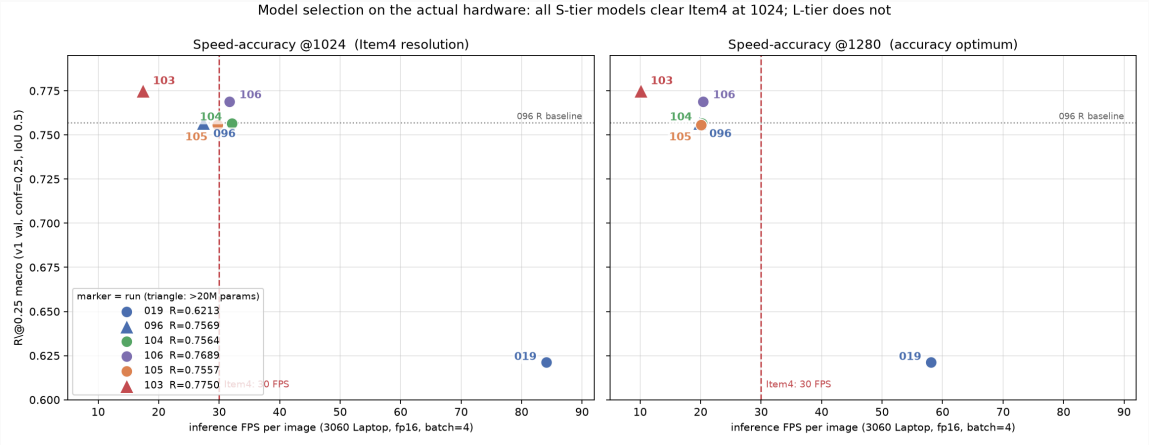


左：分辨率杠杆（1280→1024）在同 batch 下提速 1.38-1.72×（103 1.72×、104 1.59×、096 1.38×）。

右：批量杠杆（1→4）提速 1.2-3.2×；端到端（含预处理）与纯前向的差距 = 流水线开销（DEIMv2 序列受原图 2048 缩放影响最大）。

取舍：分辨率换速度要付精度代价（1024 内插掉 0.5-0.7pp，消融三），批量换速度不动精度——**优先动批量**。

推理速度 · Item4 判定与交付选型



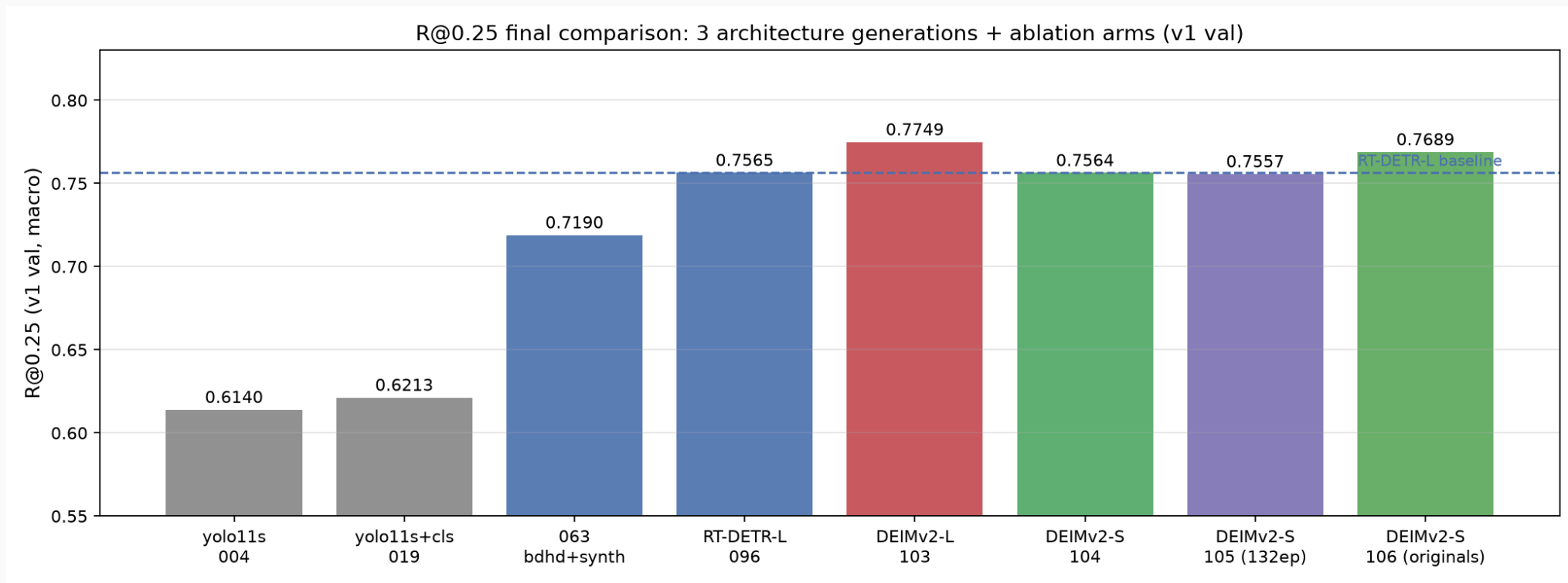
| 模型            | 1024   | 1280   |
|---------------|--------|--------|
| 019 yolo11s   | 84.1   | 58.1   |
| 096 RT-DETR-L | 27.4   | 19.8   |
| 104 DEIMv2-S  | 32.1 ✓ | 20.2   |
| 106 DEIMv2-S  | 31.7 ✓ | 20.4   |
| 105 DEIMv2-S  | 29.7   | 20.1   |
| 103 DEIMv2-L  | 17.4 ✗ | 10.1 ✗ |

3060 本机原生 batch=4 实测 (FPS/图): ≥30 的判定不再依赖跨卡折算。

104/106 (S 档) 在 1024 下达标且 R 超 RT-DETR 基线; 103 (L 档) 0.775 为精度上限但 17.4 FPS, 需 TensorRT/多流工程加速。

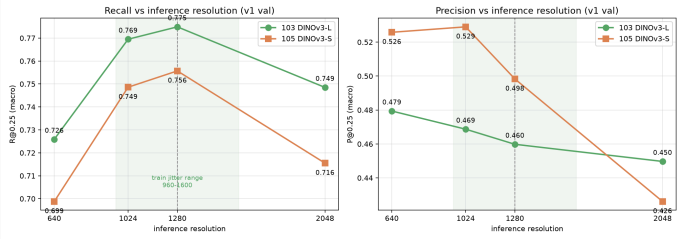
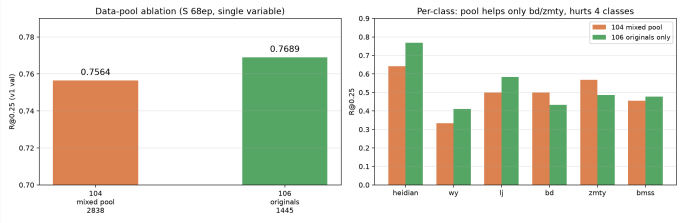
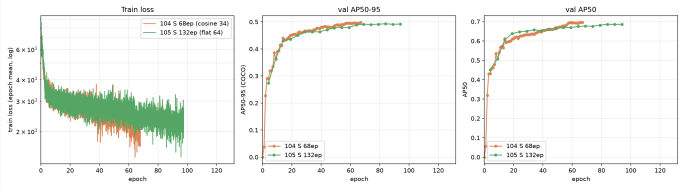
交付结论: 104 (S) 为 Item1×Item4 的帕累托点。

## R@0.25 收官对比：全系列



三代架构 + 消融臂同口径 (v1 val, conf=0.25, IoU 0.5): 103 (L) 0.775 为峰值 → 106 (S 原图-only) 0.769 → 096 (RT-DETR-L) 0.757 → 104/105 (S) 0.756 全部越过 RT-DETR 基线; 基线 yolo11s 0.614

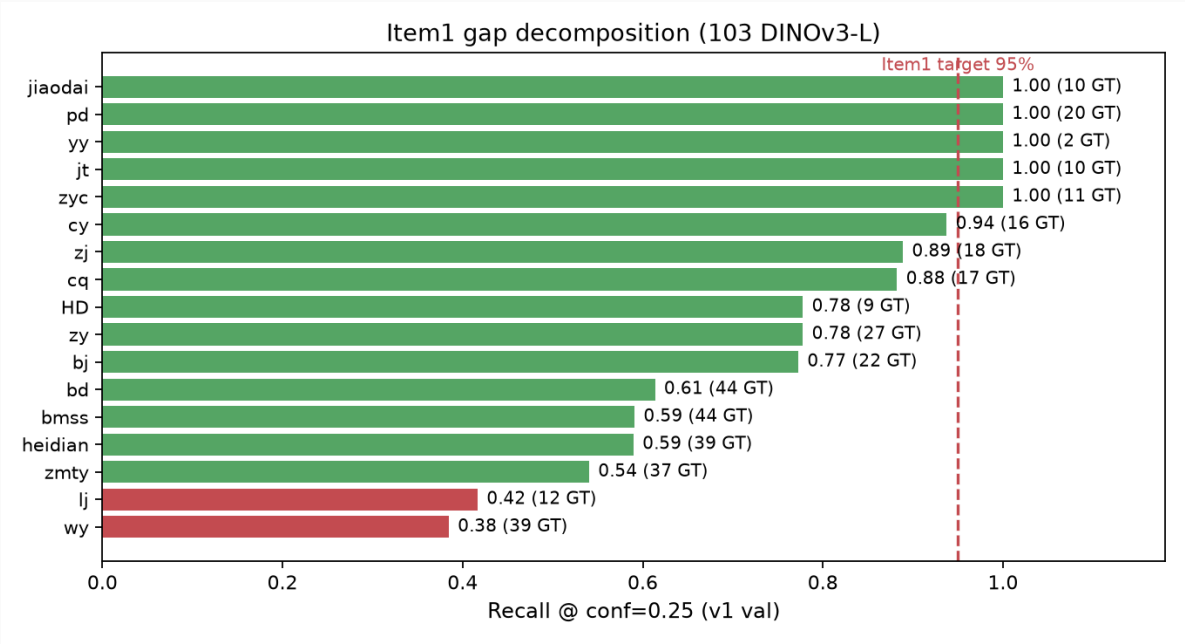
消融验证：三个单变量实验



① 训练时长：132ep 官方配方零增益（68ep 已达上限） ② 数据池：blanket 增强负收益（原图-only 反超 +1.3pp） ③ 推理尺度：倒 U 顶点锁定 1280（训练尺度）



评分差距分析：距 95% 召回还差什么



**Item1 (召回 ≥95%) 现状：**宏 maxR 0.881 (conf→0.05)，conf 0.25 下 0.775

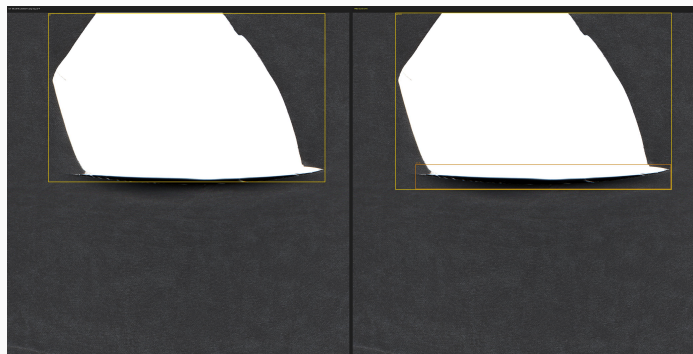
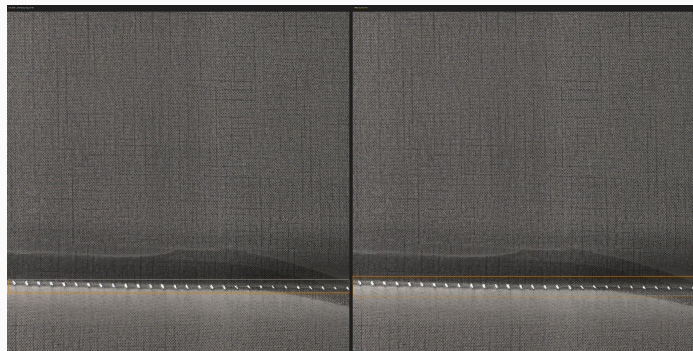
**差距来源**（按代价排序）：

- **wy 污印 0.38**：与 heidian/zmt 视觉互混——类间相似是根因
- **lj 0.42**：样本最少（12 GT）+ 与 bmss/纸张异物混淆
- **zmt 0.54**：密集碎片标注被合并检出
- **bd/heidian 低于 0.6**：小于 10px 极小目标，1280 下仅 7-9px

**共同根因：**弱纹理细粒度判别 + 标注碎片化——数据侧改良是下一轮方向

检测效果（左 GT / 右 预测， $\text{conf}=0.47$  平衡点）

检测效果（左 **GT** / 右 **预测**， $\text{conf}=\mathbf{0.47}$  平衡点）



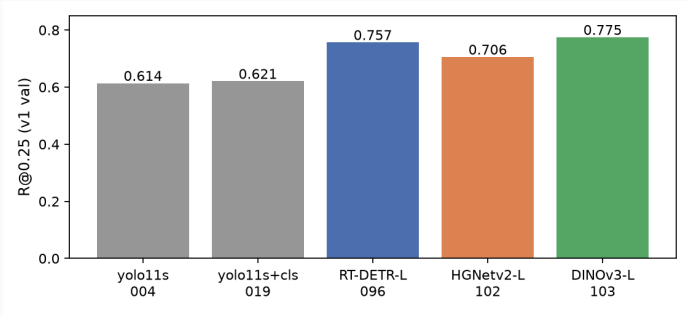
## 检测效果（左 GT / 右 预测，conf=0.47 平衡点）



jt 接头完美检出 / pd 含纸张阴影误检 / wy 小点类别混淆



# 检测效果（左 GT / 右 预测，conf=0.47 平衡点）



heidian+HD 横档对齐 / zmt y 密集印点合并现象——碎片化标注的合并是主要残差来源

## 结论

1. 数据集核心难点：**类别不均衡 × 小目标尺度 × 类间相似**
2. 数据处理：1280 对齐 + 无泄漏 8:1:1 重划；**消融证明 blanket 增强池负收益（原图-only 反超 +1.3pp）**
3. 方法迭代：YOLO 0.621 → RT-DETR-L 0.757 → **DEIMv2 DINOv3-L 0.775** (+15.4pp)
4. 交付建议：**104（DINOv3-S, R@0.25 0.756 + 3060 本机 1024 实测 32.1 FPS）精度速度双达标**，为 Item1/Item4 的最佳平衡点；103 为精度上限（需 TensorRT 过速度线）
5. 速度纪律：**口径决定结论**——batch=4 吞吐才是部署指标，单图 batch=1 延迟把 104 低估 2.7×；批量杠杆（1→4）不动精度，分辨率杠杆（1024）需付 0.5-0.7pp
6. 经验：SOTA 架构必须配官方训练配方；训练时长与推理尺度偏差均被证伪；下一轮方向 = 弱纹理类（wy/zmt）的定向数据增强